

Musikbibliotek Database – Dokumentation

Introduktion

Denna dokumentation beskriver en relationsdatabas med temat **musikbibliotek**.

Databasen är byggd för att hantera artister, album och låtar samt relationerna mellan dessa. Syftet med databasen är att visa förståelse för datamodellering, SQL (CRUD, JOINS), samt hur SQL-frågor kan motsvaras av LINQ i C# med hjälp av Entity Framework Core.

Ett ER-diagram finns som separat fil i GitHub-repot.

Datamodell och tabeller

Artist

- **Id (Primary Key)**
Ett unikt id för varje artist.
- **Name (TEXT NOT NULL, UNIQUE)**
Artistens namn. Måste vara unikt för att undvika dubletter.
- **Country (TEXT)**
Artistens ursprungsland.

TEXT NOT NULL används för Name eftersom varje artist måste ha ett namn.

Album

- **Id (Primary Key)**
Ett unikt id för varje album.
- **Title (TEXT NOT NULL)**
Albumets titel.
- **ReleaseYear (INTEGER)**
Året albumet släpptes.

- **ArtistId (Foreign Key)**
Kopplar albumet till en specifik artist.

Album är separerat från Artist för att en artist kan ha flera album (1–N-relation).

Track

- **Id (Primary Key)**
Ett unikt id för varje låt.
- **Title (TEXT NOT NULL)**
Låtens titel.
- **DurationSeconds (INTEGER NOT NULL)**
Låtens längd i sekunder.
- **AlbumId (Foreign Key)**
Kopplar låten till ett album.

Track är separerat från Album eftersom ett album kan innehålla flera låtar.

Relationer

- **Artist – 1:N – Album**
En artist kan ha flera album, men varje album tillhör endast en artist.
- **Album – 1:N – Track**
Ett album kan innehålla flera låtar, men varje låt tillhör endast ett album.

Foreign keys används för att säkerställa referensintegritet mellan tabellerna.

SQL-kommandon (Schema)

```
CREATE TABLE Artist (  
    Id INTEGER PRIMARY KEY AUTOINCREMENT,  
    Name TEXT NOT NULL UNIQUE,  
    Country TEXT  
);
```

```
CREATE TABLE Album (  
    Id INTEGER PRIMARY KEY AUTOINCREMENT,  
    Title TEXT NOT NULL,  
    ReleaseYear INTEGER CHECK (ReleaseYear >= 1900),  
    ArtistId INTEGER NOT NULL,  
    FOREIGN KEY (ArtistId) REFERENCES Artist(Id) ON DELETE CASCADE  
);
```

```
CREATE TABLE Track (  
    Id INTEGER PRIMARY KEY AUTOINCREMENT,  
    Title TEXT NOT NULL,  
    DurationSeconds INTEGER NOT NULL CHECK (DurationSeconds > 0),  
    AlbumId INTEGER NOT NULL,  
    FOREIGN KEY (AlbumId) REFERENCES Album(Id) ON DELETE CASCADE  
);
```

SQL – Exempel på frågor

Album släppta efter 2010

```
SELECT Title, ReleaseYear  
FROM Album  
WHERE ReleaseYear > 2010  
ORDER BY ReleaseYear;
```

Sortera låtar efter längd

```
SELECT Title, DurationSeconds  
FROM Track  
ORDER BY DurationSeconds DESC;
```

Hittar artister som börjar på D

```
SELECT Name  
FROM Artist  
WHERE Name LIKE 'D%';
```

Antal album per artist

```
SELECT ArtistId, COUNT(*) AS AlbumCount
FROM Album
GROUP BY ArtistId;
```

Visar låtar med album och artist (JOIN)

```
SELECT t.Title, a.Title, ar.Name
FROM Track t
JOIN Album a ON t.AlbumId = a.Id
JOIN Artist ar ON a.ArtistId = ar.Id;
```

LINQ – Exempel

Album efter 2010

```
var albumsAfter2010 = db.Albums
    .Where(a => a.ReleaseYear > 2010)
    .OrderBy(a => a.ReleaseYear)
    .Select(a => new { a.Title, a.ReleaseYear })
    .ToList();
```

JOIN mellan Track, Album och Artist

```
var trackDetails = db.Tracks
    .Select(t => new
    {
        TrackTitle = t.Title,
        AlbumTitle = t.Album.Title,
        ArtistName = t.Album.Artist.Name
    })
    .ToList();
```

GROUP BY + COUNT

```
var albumCountPerArtist = db.Albums
    .GroupBy(a => a.ArtistId)
    .Select(g => new
    {
        ArtistId = g.Key,
        AlbumCount = g.Count()
    });
```

```
})  
.ToList();
```

LINQ-frågorna motsvarar SQL-frågorna och visar hur databasen används i C# via Entity Framework Core.

Säkerhet i databaser

Säker åtkomst till databaser är viktigt eftersom databaser ofta innehåller känslig information och är ett vanligt mål för attacker. Authentication används för att säkerställa att rätt användare loggar in i systemet, till exempel via användarnamn och lösenord. Authorization avgör vilka rättigheter en användare har, till exempel om den får läsa eller ändra data. I backendprojekt används ofta principen om minsta privilegium för att begränsa åtkomst. ORM-verktyg som Entity Framework Core använder parametriserade queries, vilket minskar risken för SQL-injection. Känslig data bör aldrig lagras i klartext utan skyddas genom kryptering eller hashing. Anslutningssträngar och secrets bör hanteras säkert, till exempel via miljövariabler.

Versionshantering

Projektet versionshanterades med Git och alla SQL-filer placerades i en separat *sql*-mapp. Utvecklingen skedde i en *dev*-branch som senare mergades till *main* när funktionaliteten var färdig.

Versionshantering gör det möjligt att spåra förändringar, arbeta strukturerat och återgå till tidigare versioner vid behov.

Reflektion

Arbetet med databasen gav mig en bättre förståelse för hur relationer mellan tabeller byggs upp och används. Att skriva SQL-frågor och se hur de motsvaras av LINQ i C# var särskilt lärorikt. Det mest utmanande momentet var datamodelleringen och att bestämma vilka tabeller som skulle finnas och hur de skulle relateras till varandra. En möjlig förbättring hade varit att utöka databasen med exempelvis spellistor eller användare. Projektet visar grunderna i databasutveckling och kan enkelt byggas vidare på i framtida projekt.